

Event Sourcing - Step by Step Coding

8. Event Sourcing — Step-by-Step Coding

Since you are preparing for **4-7 years Java/Spring Boot interviews**, understand Event Sourcing in this order:

```
Event Sourcing
↓
1. Event
↓
2. Event Store
↓
3. Append events
↓
4. Replay events
↓
5. Rebuild current state
↓
6. Versioning
↓
7. Optimistic locking
↓
8. Snapshotting
↓
9. CQRS integration
```

1. What is Event Sourcing?

Normal application:

```
Create Order
↓
UPDATE orders
SET status = 'PAID'
```

```
↓
Database stores only:
status = PAID
```

We lose the complete history.

With Event Sourcing:

```
OrderCreated
↓
ItemAdded
↓
PaymentCompleted
↓
OrderShipped
```

We store **events**, not just the final state.

Current state is calculated by replaying them:

```
Event 1 → Event 2 → Event 3 → Event 4
↓         ↓         ↓         ↓
```

2. Example

Suppose order 1001 is created.

Event 1

```
{
  "orderId": 1001,
  "eventType": "ORDER_CREATED"
}
```

Event 2

```
{
  "orderId": 1001,
  "eventType": "ITEM_ADDED",
  "item": "Laptop",
  "quantity": 1
}
```

Event 3

```
{
  "orderId": 1001,
  "eventType": "PAYMENT_COMPLETED",
  "amount": 50000
}
```

Instead of storing only:

Order 1001 = PAID

we store the complete history.

3. Create Spring Boot project

Dependencies:

```
implementation 'org.springframework.boot:spring-boot-starter-web'
implementation 'org.springframework.boot:spring-boot-starter-data-jpa'

runtimeOnly 'com.mysql:mysql-connector-j'
compileOnly 'org.projectlombok:lombok'
annotationProcessor 'org.projectlombok:lombok'
```

For the basic implementation, **Kafka is not required.**

4. Create Event Store Entity

This is the most important table.

```
@Entity
@Table(
    name = "event_store",
    uniqueConstraints = {
        @UniqueConstraint(
            name = "uk_aggregate_version",
            columnNames = {"aggregate_id", "version"}
        )
    }
)
```

```

    }
)
@Getter
@Setter
@NoArgsConstructor
public class EventStoreEntry {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "aggregate_id", nullable = false)
    private Long aggregateId;

    @Column(name = "event_type", nullable = false)
    private String eventType;

    @Column(columnDefinition = "TEXT", nullable = false)
    private String payload;

    @Column(nullable = false)
    private Long version;

    @Column(nullable = false)
    private Instant timestamp;
}

```

Database:

```

event_store
-----
id
aggregate_id
event_type
payload
version
timestamp
-----
1  1001  ORDER_CREATED      {...}    1
2  1001  ITEM_ADDED         {...}    2
3  1001  PAYMENT_COMPLETED  {...}    3

```

Notice:

Events are append-only.

We don't update event #1.

5. Create Repository

```

public interface EventStoreRepository
    extends JpaRepository<EventStoreEntry, Long> {

    List<EventStoreEntry>
        findByAggregateIdOrderByVersionAsc(Long aggregateId);
}

```

This retrieves events in the correct order.

```

Version 1
  ↓
Version 2
  ↓
Version 3

```

6. Create Events

Create a common interface:

```
public interface DomainEvent {  
  
    Long getOrderId();  
}
```

OrderCreatedEvent

```
public record OrderCreatedEvent(  
    Long orderId,  
    String customerName  
) implements DomainEvent {  
  
    @Override  
    public Long getOrderId() {  
        return orderId;  
    }  
}
```

ItemAddedEvent

```
public record OrderItemAddedEvent(  
    Long orderId,  
    String itemName,  
    Integer quantity  
) implements DomainEvent {  
  
    @Override  
    public Long getOrderId() {  
        return orderId;  
    }  
}
```

PaymentCompletedEvent

```
public record PaymentCompletedEvent(  
    Long orderId,  
    Double amount  
) implements DomainEvent {  
  
    @Override  
    public Long getOrderId() {  
        return orderId;  
    }  
}
```

7. Create Order Aggregate

The aggregate represents the current state.

```
@Getter  
public class Order {  
  
    private Long id;  
  
    private String customerName;  
  
    private String itemName;  
  
    private Integer quantity;
```

```

        private Double paidAmount;

        private String status;
    }

```

8. Apply Events

This is the heart of Event Sourcing.

```

public void apply(DomainEvent event) {

    if (event instanceof OrderCreatedEvent e) {

        this.id = e.orderId();
        this.customerName = e.customerName();
        this.status = "CREATED";

    } else if (event instanceof OrderItemAddedEvent e) {

        this.itemName = e.itemName();
        this.quantity = e.quantity();

    } else if (event instanceof PaymentCompletedEvent e) {

        this.paidAmount = e.amount();
        this.status = "PAID";

    }
}

```

So:

```

ORDER_CREATED
  ↓
status = CREATED

ITEM_ADDED
  ↓
item = Laptop

PAYMENT_COMPLETED
  ↓
status = PAID

```

9. Rebuild Order From Events

Now suppose the application restarts.

We don't need to store the complete current state separately.

We can reconstruct it:

```

@Service
@RequiredArgsConstructor
public class OrderAggregateService {

    private final EventStoreRepository eventStoreRepository;

    private final ObjectMapper objectMapper;

    public Order getOrder(Long orderId) {

        List<EventStoreEntry> events =
            eventStoreRepository

        .findByAggregateIdOrderByVersionAsc(orderId);
    }
}

```

```

        Order order = new Order();

        for (EventStoreEntry entry : events) {

            DomainEvent event =
                deserialize(entry);

            order.apply(event);
        }

        return order;
    }
}

```

Conceptually:

Database

```

ORDER_CREATED
  ↓
ITEM_ADDED
  ↓
PAYMENT_COMPLETED
  ↓
  replay
  ↓
  Order
  ↓
status = PAID
item = Laptop
amount = 50000

```

10. Deserialize Events

Because the database stores JSON, we need to convert JSON back to the correct event class.

```

private DomainEvent deserialize(
    EventStoreEntry entry) {

    try {

        return switch (entry.getEventType()) {

            case "ORDER_CREATED" ->
                objectMapper.readValue(
                    entry.getPayload(),
                    OrderCreatedEvent.class
                );

            case "ITEM_ADDED" ->
                objectMapper.readValue(
                    entry.getPayload(),
                    OrderItemAddedEvent.class
                );

            case "PAYMENT_COMPLETED" ->
                objectMapper.readValue(
                    entry.getPayload(),
                    PaymentCompletedEvent.class
                );

            default ->
                throw new IllegalArgumentException(
                    "Unknown event: "
                        + entry.getEventType()
                );
        };
    }
}

```

```

        );
    };

    } catch (Exception e) {

        throw new RuntimeException(
            "Unable to deserialize event", e);
    }
}

```

11. Append Event

Now let's create an event.

```

public void createOrder(
    Long orderId,
    String customerName) {

    OrderCreatedEvent event =
        new OrderCreatedEvent(
            orderId,
            customerName
        );

    appendEvent(
        orderId,
        "ORDER_CREATED",
        event
    );
}

```

Common append method:

```

private void appendEvent(
    Long aggregateId,
    String eventType,
    DomainEvent event) {

    try {

        List<EventStoreEntry> events =
            eventStoreRepository
                .findByAggregateIdOrderByVersionAsc(
                    aggregateId
                );

        long nextVersion = events.size() + 1;

        EventStoreEntry entry =
            new EventStoreEntry();

        entry.setAggregateId(aggregateId);
        entry.setEventType(eventType);

        entry.setPayload(
            objectMapper.writeValueAsString(event)
        );

        entry.setVersion(nextVersion);
        entry.setTimestamp(Instant.now());

        eventStoreRepository.save(entry);

    } catch (Exception e) {

        throw new RuntimeException(
            "Unable to save event", e);
    }
}

```

```
}  
}
```

12. Add Item

```
public void addItem(  
    Long orderId,  
    String itemName,  
    Integer quantity) {  
  
    OrderItemAddedEvent event =  
        new OrderItemAddedEvent(  
            orderId,  
            itemName,  
            quantity  
        );  
  
    appendEvent(  
        orderId,  
        "ITEM_ADDED",  
        event  
    );  
}
```

13. Complete Payment

```
public void completePayment(  
    Long orderId,  
    Double amount) {  
  
    PaymentCompletedEvent event =  
        new PaymentCompletedEvent(  
            orderId,  
            amount  
        );  
  
    appendEvent(  
        orderId,  
        "PAYMENT_COMPLETED",  
        event  
    );  
}
```

Now database becomes:

aggregate_id	event_type	version
1001	ORDER_CREATED	1
1001	ITEM_ADDED	2
1001	PAYMENT_COMPLETED	3

14. Create Controller

```
@RestController  
@RequestMapping("/orders")  
@RequiredArgsConstructor  
public class OrderController {  
  
    private final OrderAggregateService orderService;  
  
    @PostMapping("/{id}")  
    public ResponseEntity<String> createOrder(  

```



```

        @PathVariable Long id,
        @RequestParam String customerName) {

    orderService.createOrder(
        id,
        customerName
    );

    return ResponseEntity.ok(
        "Order created"
    );
}

@PostMapping("/{id}/item")
public ResponseEntity<String> addItem(
    @PathVariable Long id,
    @RequestParam String item,
    @RequestParam Integer quantity) {

    orderService.addItem(
        id,
        item,
        quantity
    );

    return ResponseEntity.ok(
        "Item added"
    );
}

@PostMapping("/{id}/payment")
public ResponseEntity<String> payment(
    @PathVariable Long id,
    @RequestParam Double amount) {

    orderService.completePayment(
        id,
        amount
    );

    return ResponseEntity.ok(
        "Payment completed"
    );
}

@GetMapping("/{id}")
public ResponseEntity<Order> getOrder(
    @PathVariable Long id) {

    return ResponseEntity.ok(
        orderService.getOrder(id)
    );
}
}

```

15. Test the complete flow

Step 1 — Create order

POST /orders/1001?customerName=Moni

Event:

ORDER_CREATED

Step 2 — Add item

POST /orders/1001/item?item=Laptop&quantity=1

Event:

ITEM_ADDED

Step 3 — Payment

POST /orders/1001/payment?amount=50000

Event:

PAYMENT_COMPLETED

Step 4 — Get order

GET /orders/1001

The application does:

Read events

↓

ORDER_CREATED

↓

ITEM_ADDED

↓

PAYMENT_COMPLETED

↓

Replay

↓

Current Order

Response:

```
{
  "id": 1001,
  "customerName": "Moni",
  "itemName": "Laptop",
  "quantity": 1,
  "paidAmount": 50000,
  "status": "PAID"
}
```

16. Optimistic Concurrency

This is an important **4-7 year interview question**.

Suppose two requests simultaneously modify Order 1001.

Both read:

Current version = 3

Request A:

version 3 → version 4

Request B:

version 3 → version 4

We cannot allow both.

Use:

aggregateId + version

as a unique constraint:

```
@UniqueConstraint(  
    name = "uk_aggregate_version",  
    columnNames = {  
        "aggregate_id",  
        "version"  
    }  
)
```

So only one event can become:

1001 + version 4

The other request fails.

This prevents concurrent updates from silently overwriting each other.

17. Why Snapshotting?

Imagine an account has:

1,000,000 events

Every time we load it:

Event 1
Event 2
Event 3
...
Event 1,000,000

↓

Replay everything

That's expensive.

Instead:

Events 1 → 10,000

↓

Snapshot

↓

Current State at version 10,000

Events 10,001 → 10,500

↓

Replay only these

Architecture:

Event Store

|
|— Event 1
|— Event 2
|— ...
|— Event 10,000

↓

Snapshot

↓

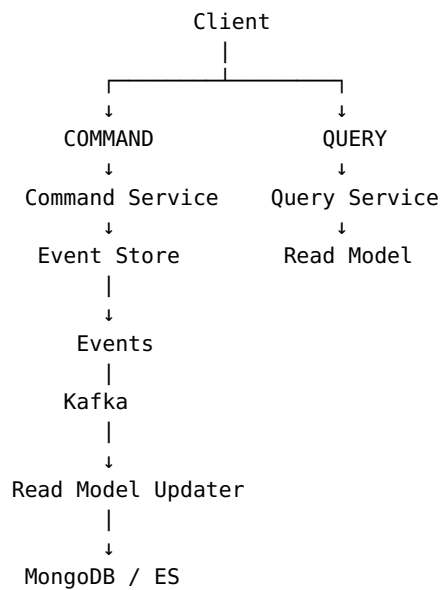
Version 10,000

↓

Replay newer events

18. Event Sourcing + CQRS

These two are often used together.



So:

CQRS
↓
separates WRITE and READ

Event Sourcing
↓
stores WRITE history as EVENTS

They are **related but not the same thing**.

19. Event Sourcing vs Audit Table

Interview question:

Audit table

orders

id
status
amount

Audit:

order_audit

old_status
new_status
changed_by
changed_at

The main database still stores the current state.

Event Sourcing

event_store

ORDER_CREATED
ITEM_ADDED

PAYMENT_COMPLETED
ORDER_SHIPPED

The events are the **source of truth**, and state is derived from them.

That's the important difference.

20. When NOT to use Event Sourcing

Don't use it simply because it sounds advanced.

Avoid it for:

Simple CRUD
↓
Employee management
↓
Basic product CRUD
↓
Simple admin application

Because it introduces:

Event versioning
Event schema evolution
Replay
Snapshotting
Storage growth
Concurrency handling
Event debugging
Data migration
Eventual consistency
Operational complexity

Use it when you genuinely need things such as:

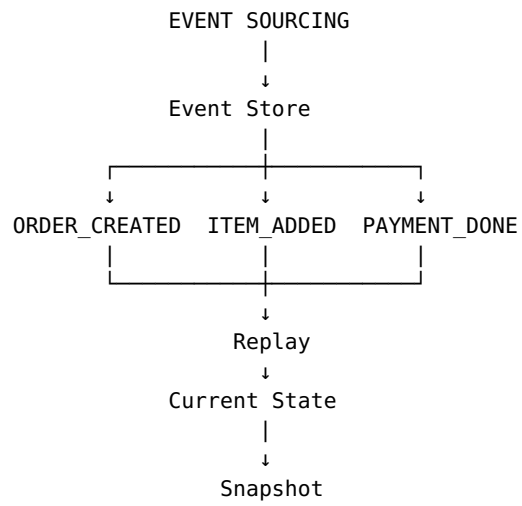
- ✓ Complete business history
 - ✓ Auditability
 - ✓ Temporal queries
 - ✓ Rebuilding read models
 - ✓ Complex domain state transitions
 - ✓ Event-driven architecture
-

21. Interview answer

For your experience level, you can say:

Event Sourcing is a pattern where we store the sequence of domain events as the source of truth instead of storing only the current state. The current state is reconstructed by replaying those events. For example, an order may have OrderCreated, ItemAdded, PaymentCompleted and OrderShipped events. I would use an append-only event store with aggregate ID and version for optimistic concurrency. For long-lived aggregates, I would use snapshots to avoid replaying millions of events. Event Sourcing is often combined with CQRS, where events are projected into optimized read models. I would not use Event Sourcing for simple CRUD applications because of the additional complexity around event versioning, replay, storage growth and eventual consistency.

Remember this diagram



One-line interview definition:

Event Sourcing = Events are the source of truth; current state is derived by replaying those events.